Geopandas



python.exposed
Learn Python to Automate, Analyze, Accelerate.

Lesmateriaal

- Via GitHub account:
- git clone https://github.com/oclaerbout/syntra_data_scientist.git
- © 2024 Olivier Claerbout
- *Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!*

Coördinatenreferentiesystemen (CRS)

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Kaartprojectie

- Wereldbollen behouden de vorm van de aarde… maar zijn niet echt handig
  - Omwille van vorm enkel op heel kleine schaal te gebruiken (1:100.10^6)
  - Meestal hebben kaart datasets een schaal van 1:250 000 of groter
- Daarom zijn er kaartprojecties: *representaties van de realiteit* waarbij elk type representatie voor- en nadelen heeft
- Het verband tussen de 2D, geprojecteerde kaart in je GIS* en de werkelijke locaties op aarde wordt bepaald door een ***coördinatenreferentiesysteem (CRS)***
- Welke kaartprojectie en welk CRS te gebruiken zijn hangt af van:
  - de regionale omvang van het gebied waarin je wilt werken
  - de analyse die je wilt uitvoeren
  - de beschikbaarheid van gegevens.

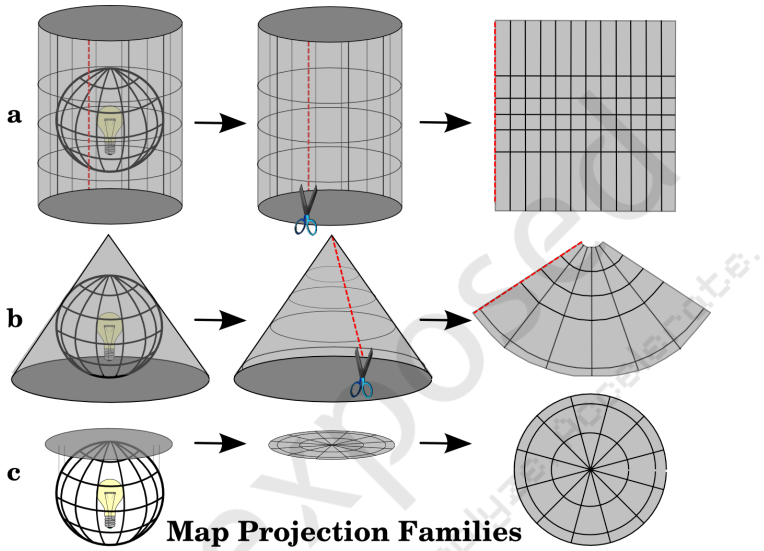
** Geografisch Informatie Systeem*

SYNTRA ANTWERPEN & VLAAMS-BRABANT

learn Python to automate, analyze, accelerate.

Drie soorten kaartprojecties

- Cilindrische projecties
- Conische projecties
- Planar (vlakke) projecties



Coördinatenreferentiesysteem (CRS)

Een CRS zorgt ervoor dat we een 3D-aarde correct kunnen weergeven op een 2D-kaart

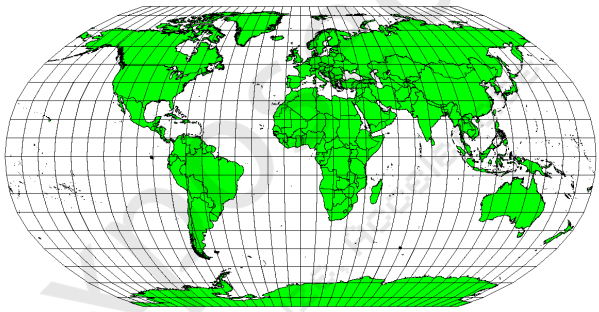
Een CRS:

- Bepaalt hoe een tweedimensionale kaart verbonden is met de echte wereld.
- Bevat informatie over welke methode wordt gebruikt om de bolvormige aarde af te beelden op een plat vlak, zoals een **kaartprojectie** (hoe breedte- en lengtegraden worden afgebeeld).
- Combineert een coördinatensysteem (zoals breedte- en lengtegraden) met een projectiemethode, zodat je de aarde op een platte kaart kunt tonen.

De keuze van de projectiemethode hangt af van de specifieke vereisten van de kaart, zoals nauwkeurigheid van **hoeken**, **afstanden** of **oppervlakken**.

Nauwkeurigheid van kaartprojecties

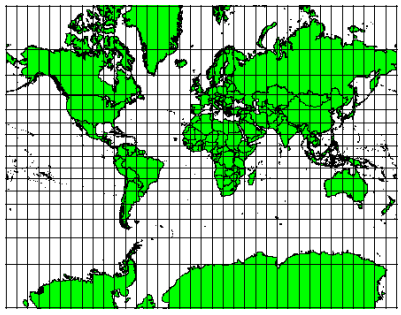
- Geen enkele projectie is volledig nauwkeurig.
- Er zijn altijd vervormingen in hoeken, afstanden of oppervlakten.



Robinson projectie, vaak gebruikt om wereldkaarten te visualiseren. De afwijkingen zijn acceptabel.

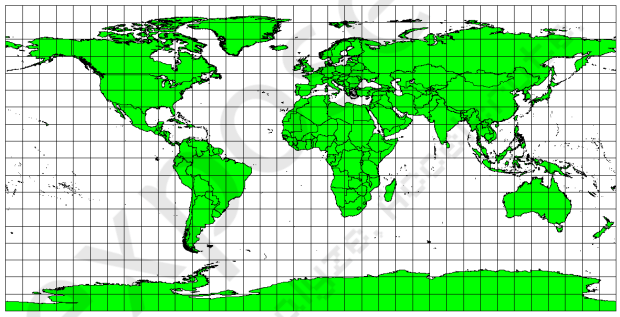
Kaartprojecties met correcte hoeken (conformale projectie)

- Conformale projecties **behouden hoeken**, zoals de 90 graden tussen Noord, Oost, Zuid en West op een globe.
- Wordt gebruikt voor navigatie en meteorologie, waar nauwkeurige hoeken belangrijk zijn.
- Nadeel: Conformale projecties **vervormen oppervlakten**, vooral bij grotere gebieden.
- Voorbeelden:
  - *Mercator-projectie*
  - *Lambert Conformale Conische projectie*



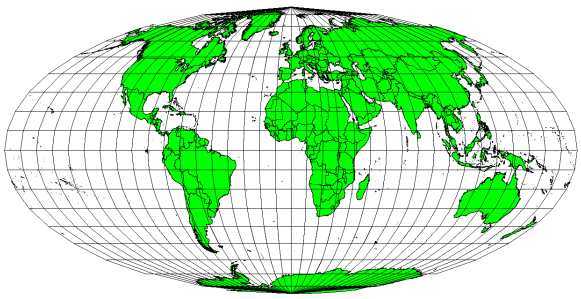
Afstandsgetrouwe Projecties

- Afstandsgetrouwe projecties **behouden afstanden** nauwkeurig, vooral van het midden van de projectie naar andere punten.
- Wordt gebruikt voor toepassingen zoals radio-, seismische- en navigatiekaarten.
- Kenmerk: de schaal van de kaart blijft constant.



Oppervlaktegetrouwe Projecties

- Oppervlaktegetrouwe projecties behouden de **juiste verhoudingen** tussen de afgebeelde gebieden en de werkelijke gebieden op aarde.
- Ideaal voor kaarten waarbij **oppervlakteberekeningen** belangrijk zijn, zoals bij het analyseren van ruimte voor bouwprojecten.
- Nadeel: Ze **vervormen hoeken**, vooral bij grotere gebieden.



Kaartprojecties

- Kaartprojectie is complex: er zijn honderden projecties wereldwijd, elk met als doel een deel van de aarde zo nauwkeurig mogelijk weer te geven.
- De keuze van de projectie wordt vaak voor je gemaakt.
- Landen gebruiken vaak standaardprojecties die gevolgd worden bij het uitwisselen van gegevens.

Python.exposed
Learn Python to Automate, Analyze, Accelerate.

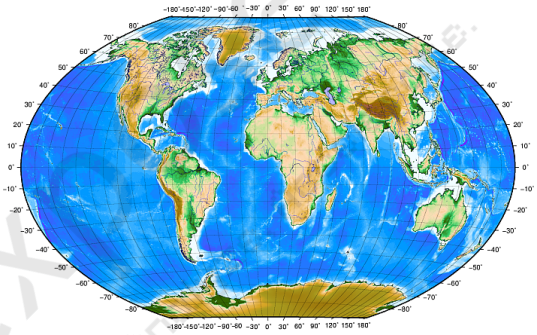
Coördinatenreferentiesystemen

- Een Coördinatenreferentiesysteem (CRS) helpt ons om elke locatie op aarde te beschrijven met behulp van coördinaten.
  - X-waarde: Geeft de oost-west positie aan.
  - Y-waarde: Geeft de noord-zuid positie aan.
  - Z-waarde: Soms wordt ook de hoogte aangegeven.

- Er zijn twee soorten CRS:
  - Geografisch Coördinatensysteem (GCS): gebaseerd op een 3D-model van de aarde. Het gebruikt breedtegraad en lengtegraad (hoekmaten) om locaties aan te geven, gemeten in graden, minuten en seconden.
  - Geprojecteerd Coördinatensysteem (PCS): neemt de 3D-vorm van de aarde en projecteert deze op een plat vlak, zoals een kaart. Het gebruikt lineaire eenheden zoals meters of kilometers om afstanden te meten. Een PCS is handig voor het nauwkeurig weergeven van afstanden, hoeken, en oppervlakten op een platte kaart, bijvoorbeeld bij het afdrukken van kaarten of voor gebruik op een computerscherm.

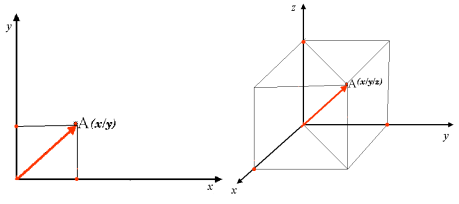
Geografische Coördinatensystemen

- **Breedtegraden:** Lijnen die evenwijdig aan de evenaar lopen (van 0° tot 90°).
- **Lengtegraden:** Lijnen die van de noordpool naar de zuidpool lopen via de nulmeridiaan (in Greenwich, 0°).
- **WGS 84**: Dit is het meest gebruikte systeem wereldwijd om geografische coördinaten aan te geven.
- Graden kunnen verder worden opgesplitst in minuten (') en seconden (") om preciezere locaties op de kaart aan te geven.
- GCS gebruikt dus breedte- en lengtegraden om te zeggen waar iets precies op de aarde is.



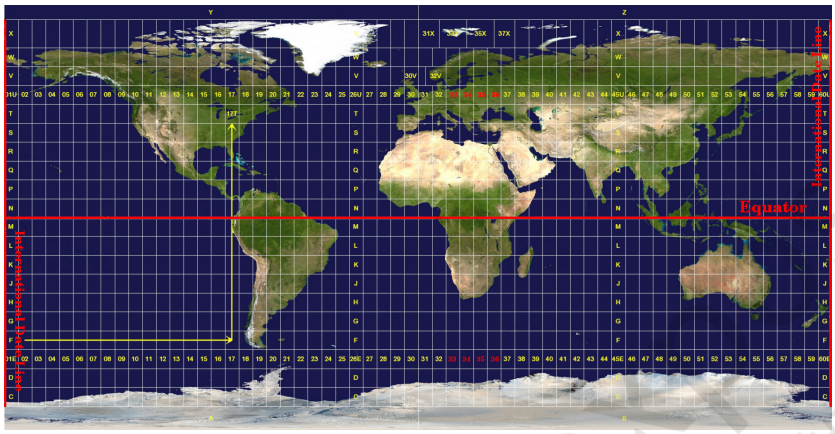
Geprojecteerde Coördinatenreferentiesystemen

- Een Geprojecteerd Coördinatensysteem (PCS) helpt om een kaart plat te maken.
- In een 2D-systeem heb je twee assen: X (horizontaal) en Y (verticaal) die elkaar kruisen.
- In een 3D-systeem komt er een extra as bij, de Z-as, voor hoogte (dus je meet ook hoe hoog iets is).
- Op het noordelijk halfrond:
  - Y gaat omhoog naar het noorden.
  - X gaat naar rechts, naar het oosten.
- Op het zuidelijk halfrond:
  - Y gaat naar beneden, naar het zuiden.
  - X gaat naar links, naar het westen.
- Een veelgebruikte projectie is Universal Transverse Mercator (UTM), die helpt om de bolle aarde op een plat vlak te tekenen.



Universal Transverse Mercator (UTM)

- UTM-projectie: Wereldwijd gebruikt, verdeelt de aarde in 60 zones van elk 6 graden breed.



EPSG Codes

EPSG-codes zijn unieke, numerieke identificaties voor specifieke coördinatensystemen, ontwikkeld door de *European Petroleum Survey Group* (EPSG). Elke code verwijst naar een bepaald CRS en bevat informatie over:

- Het type coördinatensysteem (GCS of PCS),
- De gebruikte projectiemethode,
- De eenheden van meting (zoals meters of graden),
- Andere technische details die nodig zijn om geografische data correct te interpreteren.

Dit is cruciaal in tools zoals GeoPandas: GeoPandas maakt gebruik van EPSG-codes om het juiste CRS toe te wijzen aan geografische data. Dit zorgt ervoor dat alle punten, lijnen of polygonen op de juiste plaats en in de juiste schaal op de kaart worden weergegeven.

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:4326 (WGS 84)

- Dit is de standaard CRS die wordt gebruikt voor geografische coördinaten wereldwijd. Het wordt vaak gebruikt door **GPS-systemen**.
- Eenheden: Graden (latitude, longitude).
- Gebruik: *Geschikt voor het visualiseren van locaties met coördinaten in breedte- en lengtegraden, maar niet voor het nauwkeurig berekenen van afstanden of oppervlakten omdat het geen geprojecteerd systeem is.*
- Voorbeeld: Globale locaties tonen op basis van coördinaten (zoals GPS-gegevens).

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Python.exposed
Learn Python to Automate, Analyze, Accelerate.

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:4326 (WGS 84) – Sidenote:

- De locatie-informatie die je GPS-apparaat ontvangt en weergeeft is in EPSG:4326. Dit betekent dat de exacte locatie van waar je bent (bijvoorbeeld: breedtegraad en lengtegraad) in graden wordt weergegeven en door het apparaat wordt gebruikt om je positie te bepalen. Dit is hoe GPS-systemen jouw huidige positie op de aarde vaststellen.
- De achtergrondkaarten die je ziet op je GPS-scherm (zoals wegenkaarten, stadskaarten, enzovoort) worden bijna altijd weergegeven in EPSG:3857 (Web Mercator). Dit komt omdat EPSG:3857 een platte projectie van de bolvormige aarde is, en het maakt het mogelijk om kaarten op een plat scherm (zoals dat van een GPS-apparaat, smartphone of computer) weer te geven.
- Dus wat je ziet op het scherm is waarschijnlijk een kaart in EPSG:3857, terwijl je GPS de locatie van je positie berekent in EPSG:4326 en die positie vervolgens op de kaart projecteert.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:3857 (Web Mercator)

- Beschrijving: Dit is een geprojecteerd CRS en wordt vaak gebruikt voor webmapping applicaties, zoals Google Maps en OpenStreetMap.
- Eenheden: Meters.
- Gebruik: Geschikt voor het weergeven van kaarten in een online context. Het vervormt afstanden en gebieden, vooral dichter bij de polen, maar is handig voor webmapping omdat het compatibel is met vele internetkaartdiensten.
- Voorbeeld: Kaarten tonen in een webbrowser.

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:326xx of EPSG:327xx (UTM Zones)

- Beschrijving: Het Universal Transverse Mercator (UTM) systeem verdeelt de wereld in zones, waarbij elk een specifieke EPSG-code heeft (EPSG:326xx voor noordelijk halfrond, EPSG:327xx voor zuidelijk halfrond, waarbij xx staat voor de UTM-zone).
- Eenheden: Meters.
- Gebruik: Geschikt voor nauwkeurige afstand- en oppervlakteberekeningen binnen een specifieke zone, zoals het meten van afstanden of oppervlakten in lokale regio's.
- Voorbeeld: Nauwkeurige landmeting of het plannen van infrastructuur.

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:3035 (ETRS89 / LAEA Europe)

- Beschrijving: Dit is een Equal Area projectie speciaal *ontworpen voor het Europese continent*. Het behoudt de oppervlakten nauwkeurig.
- Eenheden: Meters.
- Gebruik: Geschikt voor oppervlakteberekeningen en kaarten van heel Europa zonder grote vervormingen.
- Voorbeeld: Het berekenen van het oppervlak van landen in Europa (zoals België).

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:3413 (NSIDC Sea Ice Polar Stereographic North)

- Beschrijving: Dit CRS is geprojecteerd om nauwkeurige berekeningen in poolgebieden mogelijk te maken, zoals Arctica en Antarctica.
- Eenheden: Meters.
- Gebruik: Geschikt voor het werken met pooldata of voor het meten van afstanden en oppervlakten in de poolregio's.
- Voorbeeld: Het analyseren van poolgebieden, zoals zeeniveaumetingen in het Noordpoolgebied.

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Veelgebruikte EPSG-codes en hun toepassingen

EPSG:6933 (World Cylindrical Equal Area)

- Beschrijving: Een Equal Area projectie die wereldwijd wordt gebruikt en vervorming van oppervlakten minimaliseert.
- Eenheden: Meters.
- Gebruik: Voor mondiale kaarten die geen vervorming van de oppervlakte willen hebben.
- Voorbeeld: Wereldwijde ecologische analyses, zoals het berekenen van het oppervlak van continenten.

Veelgebruikte EPSG-codes en hun toepassingen

Hoe kies je de juiste EPSG?

- Globale weergave van punten (bijv. breedte- en lengtegraadgegevens): Gebruik EPSG:4326 (WGS 84).
- Webmapping en visuele kaarten op het internet: Gebruik EPSG:3857 (Web Mercator).
- Nauwkeurige afstand- en oppervlakteberekeningen in kleine gebieden: Gebruik UTM zones (EPSG:326xx of EPSG:327xx).
- Oppervlakteberekeningen voor Europa: Gebruik EPSG:3035 (ETRS89 / LAEA Europe).
- Werken met poolgebieden: Gebruik EPSG:3413 voor het noordpoolgebied of een vergelijkbare projectie voor het zuidpoolgebied.
- Mondiale oppervlakteberekeningen zonder vervorming: Gebruik EPSG:6933 (World Cylindrical Equal Area).

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Geopandas

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Resources

- https://geopandas.org/en/stable/getting_started/install.html
- https://toarches.medium.com/mapping-coronavirus-b89880ecd400
- https://medium.com/@haniszulaikha/starter-tutorial-on-geopandas-d3ede5cc16e9
- https://medium.com/@kylembandy/the-perfect-day-at-walt-disney-world-finding-an-optimal-route-through-magic-kingdom-using-python-f4fe2cc104b8

SYNTRA ANTWERPEN & VLAAMS-BRABANT

python.exposed
learn Python to automate, analyze, accelerate.

Shapely

- Shapely is een Python-bibliotheek die gebruikt wordt voor het manipuleren en analyseren van geometrische objecten zoals punten, lijnen, en polygonen.
- Shapely maakt het mogelijk om verschillende geometrische operaties op geografische objecten uit te voeren.
- Je kan ook geometrische objecten maken, vb basisvormen zoals:
  - *Punten (locaties met een enkele coördinaat)*
  - *Lijnen (een reeks van verbonden punten)*
  - *Polygonen (gesloten vormen zoals een driehoek of een vierkant)*

```python
from shapely.geometry import Point, LineString, Polygon

# Een punt maken
point = Point(1, 1)

# Een lijn maken
line = LineString([(0, 0), (1, 1), (2, 2)])

# Een polygoon maken
polygon = Polygon([(0, 0), (1, 0), (1, 1), (0, 1)])
```


Wie is Fiona?

- Fiona is een Python-bibliotheek die speciaal is ontworpen voor het lezen en schrijven van geografische bestanden in verschillende bestandsformaten, zoals Shapefiles, GeoJSON, en andere GIS-gerelateerde formaten.
- Fiona fungeert als een brug tussen Python en geografische informatiebestanden, en het maakt het eenvoudig om geografische data te beheren zonder complexe GIS-software.
- Fiona ondersteunt een breed scala aan geografische bestandsformaten zoals:
  - Shapefile (.shp)
  - GeoJSON (.geojson)
  - KML (.kml)
  - GPKG (GeoPackage)

```python
import fiona

# Inlezen van een shapefile
with fiona.open('mijn_data.shp') as shapefile:
    for feature in shapefile:
        print(feature)
```

GeoPandas

GeoPandas is een Python-bibliotheek die het eenvoudig maakt om met geografische gegevens te werken door de mogelijkheden van de Pandas-bibliotheek uit te breiden met ondersteuning voor geospatiale data.

GeoPandas combineert:

- *de kracht van Pandas*
- *de geometrische mogelijkheden van Shapely*
- *het bestandsbeheer van Fiona*

Hierdoor kun je op een gebruiksvriendelijke manier geografische data in Python manipuleren zonder diepgaande kennis van GIS-software.

Wat doet GeoPandas?

GeoPandas maakt het mogelijk om geografische data zoals punten, lijnen, en polygonen (bijvoorbeeld locaties van steden, wegen, of grenzen van gebieden) te lezen, te manipuleren en te analyseren.

Het bouwt voort op de kracht van Pandas, maar voegt daar geometrische en geografische functies aan toe, zoals:

- *Lezen en schrijven van geografische bestanden*
- *Ruimtelijke operaties: geografische objecten manipuleren, zoals:*
  - *Intersectie: Bepalen waar objecten elkaar overlappen.*
  - *Buffer: Een gebied rondom een object maken (bijvoorbeeld een zone rondom een weg).*
  - *Afstand: De afstand tussen twee geografische objecten berekenen.*

Wat doet GeoPandas?

- *Visualisatie:*
  - *GeoPandas maakt het eenvoudig om geografische gegevens op een kaart te visualiseren door gebruik te maken van de ingebouwde plotting-mogelijkheden (op basis van Matplotlib).*
- *CRS en Projecties:*
  - *GeoPandas ondersteunt het transformeren van coördinatensystemen (CRS).*
    - *Dit is handig als je verschillende datasets hebt die in andere coördinatensystemen zijn, omdat je ze kunt converteren naar hetzelfde systeem met behulp van EPSG-codes.*

Python.exposed Learn Python to Automate, Analyze, Accelerate.

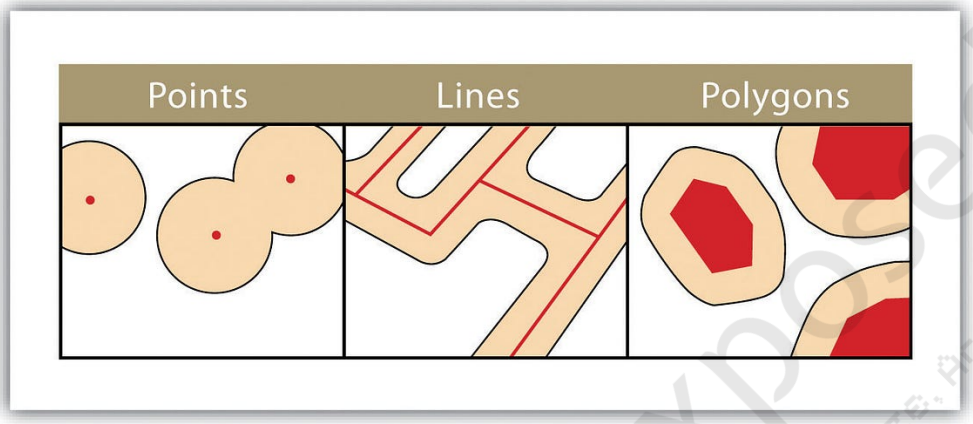
Geopandas en coördinatensystemen

Het begrijpen van coördinatensystemen helpt je om geografische data op de juiste manier te visualiseren, meten en analyseren in GeoPandas, waardoor je nauwkeurige en betrouwbare resultaten krijgt:

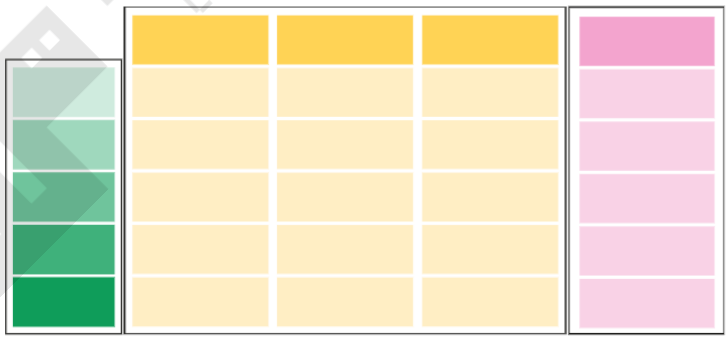
- ***Correcte Weergave van Gegevens**: GeoPandas werkt met geografische data. Zonder een goed coördinatensysteem (GCS of PCS) kunnen locaties op je kaart verkeerd worden weergegeven.*
- ***Nauwkeurige Metingen**: Als je afstanden, oppervlaktes of hoeken wilt meten, dan moet je het juiste coördinatensysteem gebruiken. Met een PCS zoals UTM, meet je in meters, waardoor je nauwkeurig kunt rekenen.*
- ***Transformaties tussen Systemen**: Geografische data kan uit verschillende bronnen komen, en die kunnen verschillende coördinatensystemen hebben. Met GeoPandas kun je deze coördinatensystemen transformeren zodat ze allemaal correct op elkaar passen.*
- ***Werken met Verschillende Kaartprojecties**: Verschillende analyses vragen om verschillende projecties. Bijvoorbeeld, als je grote gebieden wil vergelijken (zoals landen), is een andere projectie nodig dan wanneer je een klein gebied (zoals een stad) in detail wil bestuderen.*

De basis van alles

We breiden pandas uit met geografische data.



geopandas.GeoDataFrame



`index` `data` `geometry`

pandas.Series objecten

geopandas.GeoSeries objecten

python.exposed Learn Python to Automate, Analyze, Accelerate.

Data lezen & schrijven

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Data inlezen

Geodatasets maakt het eenvoudig om toegang te krijgen tot voorbeelddatasets met geografische data

```
from geodatasets import get_path
path_to_file = get_path("nybb")
```

```
gdf = geopandas.read_file(path_to_file)
gdf.head()
```

	BoroCode	BoroName	Shape_Leng	Shape_Area	geometry
0	5	Staten Island	330470.010332	1.623820e+09	MULTIPOLYGON (((970217.022 145643.332, 970227....
1	4	Queens	896344.047763	3.045213e+09	MULTIPOLYGON (((1029606.077 156073.814, 102957...
2	3	Brooklyn	741080.523166	1.937479e+09	MULTIPOLYGON (((1021176.479 151374.797, 102100...
3	1	Manhattan	359299.096471	6.364715e+08	MULTIPOLYGON (((981219.056 188655.316, 980940....
4	2	Bronx	464392.991824	1.186925e+09	MULTIPOLYGON (((1012821.806 229228.265, 101278...

Dit is de geometry kolom; er kunnen er meerdere zijn maar altijd maar 1 actieve.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Data opslaan als json

```python
from shapely.geometry import Polygon

# Definieer enkele polygonen (vierkanten in dit voorbeeld met elk punt (oosterlengte,noorderbreedte)
polygonen = [Polygon([(4, 52), (5, 52), (5, 53), (4, 53)]),  # Vierkant boven Amsterdam
             Polygon([(4, 51), (5, 51), (5, 52), (4, 52)]),  # Vierkant rond Amsterdam
             Polygon([(3, 50), (4, 50), (4, 51), (3, 51)])]  # Vierkant rond Brussel

# Maak een nieuw GeoDataFrame met deze polygonen
df_polygons = gpd.GeoDataFrame({'stad': steden, 'geometry': polygonen})
```

```python
## Opslaan van je GeoData
df_polygons.to_file('my_geodata.geojson', driver="GeoJSON")
```

Hiermee geef je aan dat je het bestand in GeoJSON-formaat wilt opslaan. Dit is een veelgebruikt formaat voor het uitwisselen van geografische gegevens.

Deze kan je ook gewoon terug inlezen:

```python
## Opnieuw lezen van je GeoData
read_file = geopandas.read_file('my_geodata.geojson')
read_file.head()
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Data opslaan als shape file

```python
## Opslaan als shapefile
df_polygons.to_file('GEODATA/my_geodata.shp')
```

/ ··· / 04 Geopandas / GEODATA /

Name	Last Modified
my_geodata.cpg	2 minutes ago
my_geodata.dbf	2 minutes ago
my_geodata.geojson	2 minutes ago
my_geodata.prj	2 minutes ago
my_geodata.shp	2 minutes ago
my_geodata.shx	2 minutes ago

Shapefiles zijn een veelgebruikt formaat voor het opslaan van geografische informatie en worden breed ondersteund door GIS-software

Deze kan je ook gewoon terug inlezen

```python
## Opnieuw inlezen van een shapefile
read_shapefile = geopandas.read_file('GEODATA/my_geodata.shp')
read_shapefile.head()
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Bewerkingen

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Geografische operaties

Zie hier voor nog veel meer opties:

https://geopandas.org/en/stable/docs/reference/geoseries.html

- Je kan een *geografische functie* uitoefenenen op een GeoDataFrame.
  - Deze zoekt de actieve GeoSeries en voert de operatie hierop uit. Voorbeelden:
    - gdf.area: berekent de oppervlakte van elk geografisch kenmerk in de GeoDataFrame (vb de wijken van New York, zoals Staten Island, ...). De uitvoer toont de opp voor elke wijk.
    - gdf.boundary: haalt de grenzen op van elk geografisch kenmerk. De uitvoer toont MULTILINESTRING-geometrieën, die de lijnen vertegenwoordigen die de buitenste grenzen van de regio's definiëren.
    - gdf.convex_hull: genereert de convexe omhulling van elk geografisch kenmerk. Dit is de kleinste convexe vorm die elk kenmerk kan insluiten. De uitvoer toont POLYGON-geometrieën die deze convexe omhullingen vertegenwoordigen.
    - gdf.centroid: Deze functie berekent het zwaartepunt (geometrisch middelpunt) van elk geografisch kenmerk. De uitvoer toont POINT-geometrieën, die de coördinaten van de zwaartepunten voor elke wijk aangeven.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Python.exposed Learn Python to Automate, Analyze, Accelerate.

Geografische operaties

```
gdf.area
```

```
0    1.623822e+09
1    3.045214e+09
2    1.937478e+09
3    6.364712e+08
4    1.186926e+09
dtype: float64
```

```
gdf.boundary
```

```
0    MULTILINESTRING ((970217.022 145643.332, 97022...
1    MULTILINESTRING ((1029606.077 156073.814, 1029...
2    MULTILINESTRING ((1021176.479 151374.797, 1021...
3    MULTILINESTRING ((981219.056 188655.316, 98094...
4    MULTILINESTRING ((1012821.806 229228.265, 1012...
dtype: geometry
```

```
gdf.convex_hull
```

```
0    POLYGON ((915517.688 120121.881, 915467.035 12...
1    POLYGON ((1000721.532 136681.776, 994611.996 2...
2    POLYGON ((988872.821 146772.032, 983670.606 14...
3    POLYGON ((977855.445 188082.322, 971830.134 19...
4    POLYGON ((1017949.978 225426.885, 1015563.562 ...
dtype: geometry
```

```
gdf.centroid
```

```
0      POINT (941639.45 150931.991)
1    POINT (1034578.078 197116.604)
2     POINT (998769.115 174169.761)
3     POINT (993336.965 222451.437)
4      POINT (1021174.79 249937.98)
dtype: geometry
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

python.exposed Learn Python to Automate, Analyze, Accelerate.

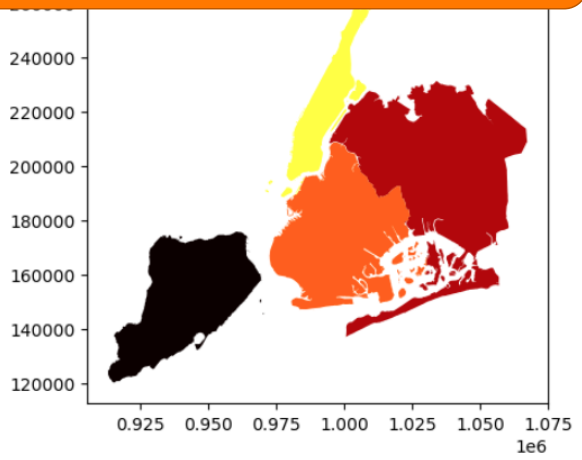
Op een kaart afbeelden

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Kaartjes plotten

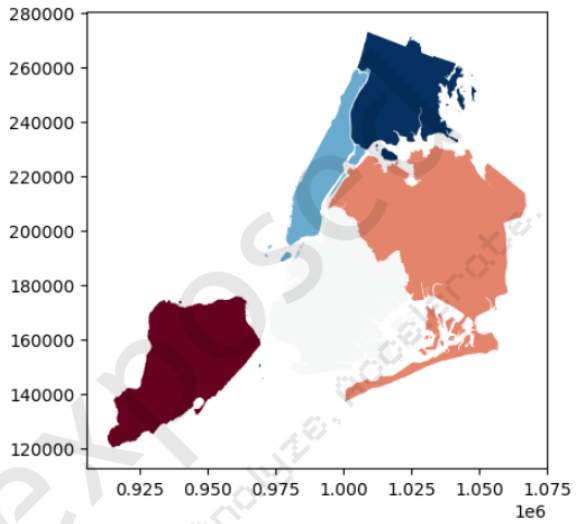
Maar 4 delen van de 5 getoond! Eentje ziet wit.. Dus geen goede colormap

Bvb RdBu is beter



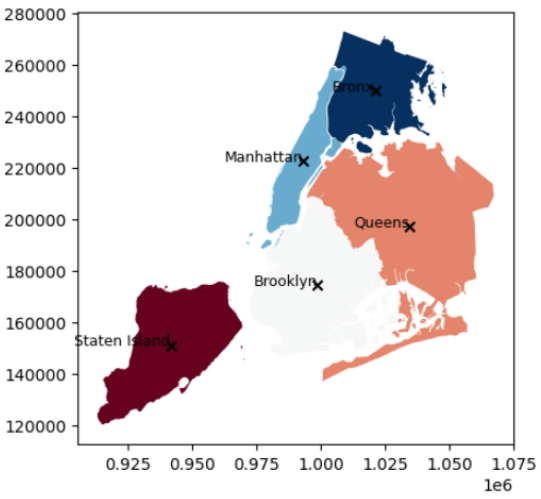

```
fig, ax = plt.subplots()
gdf.plot(cmap='RdBu', ax=ax)
```

```
<Axes: >
```



Centroids erbij plotten

Merk op: In plaats van ax.plot() moet je GeoDataFrame.plot() gebruiken en de ax meegeven als parameter om op een bestaand Matplotlib-figuur te tekenen




```
# Voeg de centroids toe als punten
fig, ax = plt.subplots()
gdf.plot(cmap='RdBu', ax=ax)

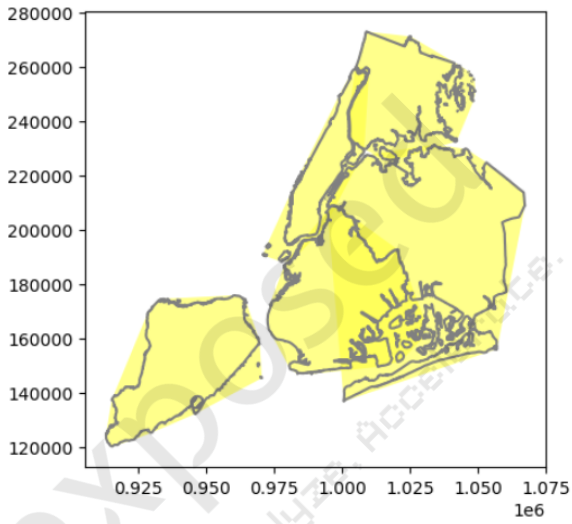
gdf_centroids = gdf['centroid']
ax.scatter(gdf_centroids.x, gdf_centroids.y, color='black', marker='x', label='Centroids')

# Optioneel: Voeg labels toe bij de centroids
for idx, row in gdf.iterrows():
    plt.text(row['centroid'].x, row['centroid'].y, row['BoroName'], fontsize=9, ha='right')
```

Convex hull & grenzen

```
# Voeg de Convex Hull toe en plot enkel de grenzen

fig, ax = plt.subplots()
# voeg de convex hull van de geometrieën toe
gdf.convex_hull.plot(color='yellow',alpha=0.5, ax=ax)
# plot enkel de grenzen van de geometrieën
gdf.boundary.plot(ax=ax, color='grey', linewidth=1.5, alpha=1)
```



Learn Python to Automate, Analyze, Accelerate.

Interactieve kaart met echte achtergrond

```
import folium

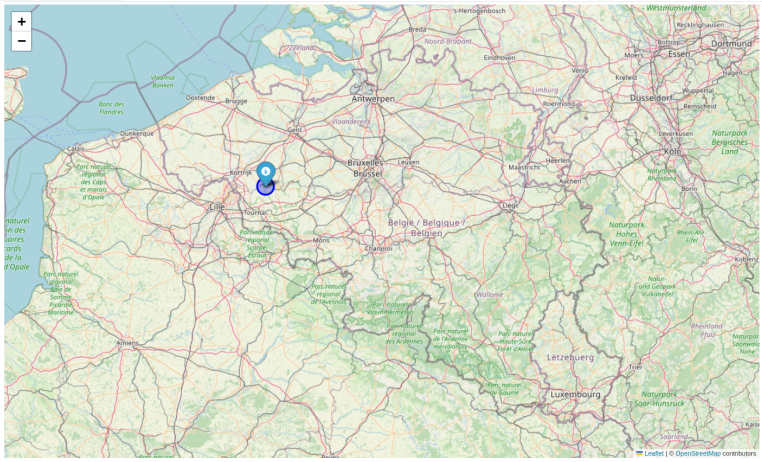
# Coördinaten van het centrum van België
belgium_coords = [50.5039, 4.4699]

# Maak een Folium-kaart, gecentreerd op België
map_belgium = folium.Map(location=belgium_coords, zoom_start=8, tiles="OpenStreetMap")

# Voeg een marker toe op basis van de gegeocodeerde coördinaten
folium.Marker(
    [location.latitude, location.longitude],
    popup=f"{address}",
    icon=folium.Icon(color="red")
).add_to(map_belgium)

# Voeg een cirkel toe rond de locatie
folium.Circle(
    location=[location.latitude, location.longitude],
    radius=5000,  # Radius in meters
    color="blue",  # Randkleur
    fill=True,
    fill_color="blue",  # Vul kleur (Hue)
    fill_opacity=0.3  # Doorzichtigheid van de cirkel
).add_to(map_belgium)

# Voeg een marker toe met Tooltip (hover effect)
tooltip_text = f"Adres: Hallo"
folium.Marker(
    [location.latitude, location.longitude],
    popup=folium.Popup(f"Adres"),
    tooltip=folium.Tooltip(tooltip_text),  # Tooltip voor hover
    icon=folium.Icon(color="blue")
).add_to(map_belgium)
# Toon de kaart
map_belgium
```



Buffers, boundaries & intersections

SYNTRA ANTWERPEN & VLAAMS-BRABANT

python.exposed Learn Python to automate, analyze, accelerate.

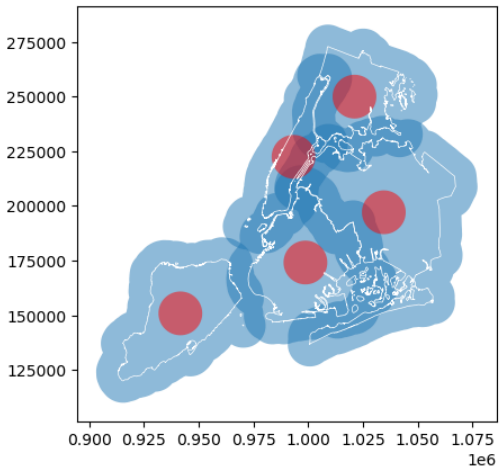
Boundaries & bufferen

```
# Define boundary
gdf["boundary"] = gdf.boundary

# buffering the active geometry by 10 000 feet (geometry is already in feet)
gdf["buffered"] = gdf.buffer(10000)

# buffering the centroid geometry by 10 000 feet (geometry is already in feet)
gdf["buffered_centroid"] = gdf["centroid"].buffer(10000)

# saving the first plot as an axis and setting alpha (transparency) to 0.5
ax = gdf["buffered"].plot(alpha=.5)
# passing the first plot as an axis to the second
gdf["buffered_centroid"].plot(ax=ax, color="red", alpha=.5)
# passing the first plot and setting linewidth to 0.5
gdf["boundary"].plot(ax=ax, color="white", linewidth=.5)
```



SYNTRA ANTWERPEN & VLAAMS-BRABANT

Intersection & within

- *intersection: overlappen of snijden twee geometrieën elkaar (gedeeltelijk of volledig)?*
- *within: ligt een geometrie volledig binnen een andere geometrie?*

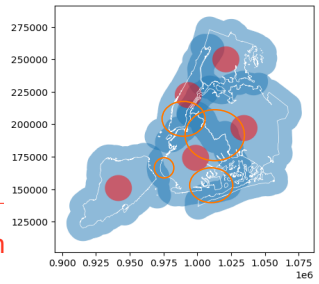
Stadsdelen van New York:




```
gdf = gdf.set_index('BoroName')
brooklyn = gdf.loc["Brooklyn", "geometry"]
brooklyn
```

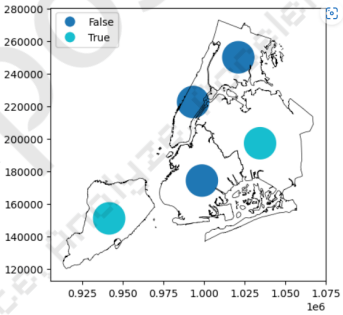
```
gdf["buffered"].intersects(brooklyn)
```

```
BoroName
Staten Island     True
Queens            True
Brooklyn          True
Manhattan         True
Bronx            False
dtype: bool
```




```
gdf["within"] = gdf["buffered_centroid"].within(gdf)
gdf["within"]
```

```
BoroName
Staten Island     True
Queens            True
Brooklyn         False
Manhattan        False
Bronx            False
Name: within, dtype: bool
```



SYNTRA ANTWERPEN & VLAAMS-BRABANT

Dank u voor uw aandacht

SYNTRA ANTWERPEN & VLAAMS-BRABANT